JEPA-LLM Tuning Framework for Combating GST and Sales Tax Input Tax Credit Fraud Pattern Detection

B.ASHRITHA
Roll No. 23K91A6732
Dept. of CSE (Data Science)
TKR College of Engineering and Technology (Autonomous)
[City, State, Country]
[email address]

B.DIVYA
Roll No. 23K91A6739
Dept. of CSE (Data Science)
TKR College of Engineering and Technology (Autonomous)
[City, State, Country]
[email address]

A.KISHORE
Roll No. 23K91A6716
Dept. of CSE (Data Science)
TKR College of Engineering and Technology (Autonomous)
[City, State, Country]
[email address]

A.YAGNESH
Roll No. 23K91A6711
Dept. of CSE (Data Science)
TKR College of Engineering and Technology (Autonomous)
[City, State, Country]
[email address]

Dr. N. Ch. Ravi
Professor in CSE (Data Science)
Dept. of CSE (Data Science)
TKR College of Engineering and Technology (Autonomous)
[City, State, Country]
[email address]

***Abstract*—Input tax credit (ITC) fraud under India's Goods and Services Tax, carried out through fake invoices, circular trading and shell companies, causes large revenue losses, yet confirmed fraud labels are scarce and most systems cannot say why a claim is flagged. This paper presents TaxSentinel, a self-supervised and explainable framework. A Joint Embedding Predictive Architecture (JEPA) learns normal taxpayer behaviour without labels; its prediction error is combined, through a noisy-OR, with circular-trading and shared-identity cluster detection and with invoice-level anomaly scores. Flagged taxpayers receive a numbered evidence list, and a large language model writes a case note in which every claim cites that evidence. On a seeded synthetic ecosystem of 600 taxpayers and 64,474 invoices, the system reaches precision@79 of 0.949, F1 0.955 and ROC-AUC 0.997, against 0.620, 0.432 and 0.876 for a rule baseline, and recovers all five injected rings. Performance at the fixed threshold is lower on other generated ecosystems, the language model is prompted rather than fine-tuned, and no real tax data was used.**

***Index Terms*—circular trading, explainable AI, GST fraud, JEPA, large language model, self-supervised learning, shell entities**

I. INTRODUCTION

The Goods and Services Tax (GST), introduced in India in 2017, lets a business set the tax it paid on purchases, its input tax credit (ITC), against the tax it collects on sales. The mechanism removes cascading taxes but turns an invoice into something that carries money, and because returns are self-assessed, an invoice for a supply that never happened can still hand a buyer a credit [1]. Three patterns dominate: fake invoicing by networks of paper firms, circular trading in which a few firms pass the same value around a loop, and shell entities that share an address, trade among themselves, pay almost no tax in cash and stop filing. Official figures cited in the base paper show detected evasion rising from Rs 1.01 lakh crore in FY 2022-23 to Rs 2.01 lakh crore in FY 2023-24, with cases booked rising from 4,872 to 6,084 [1].

Current practice combines audits after suspicion, fixed mismatch rules and, increasingly, supervised machine learning on labelled taxpayer records [2], [3], [4], [5]. Each has a cost. Audits are slow and find fraud after the revenue is lost; fixed rules are transparent and therefore easy to stay under; supervised models need confirmed cases, which are scarce, late and biased towards what investigators happened to pursue; and checks on one return or invoice cannot see a loop of five firms that each look ordinary. Few systems explain a flag in terms an officer can verify.

This paper presents **TaxSentinel**, a self-supervised, explainable framework that learns normal GST behaviour from unlabelled data, combines invoice, return-behaviour and network evidence, and explains each flag with cited evidence. Its contributions are:

1) a three-layer, 19-feature representation of taxpayer-months and a small JEPA behaviour encoder trained without labels in 11.7 s on a CPU;
2) scored circular-trading and shared-identity detectors and a noisy-OR risk that combines them with the JEPA deviation and invoice-level anomaly scores;
3) evidence-gated case notes written by a large language model, with automatic citation and grounding checks; and
4) an evaluation against a rule baseline with ablations, a cross-seed study and a transfer check, and a secured full-stack application.

Section II reviews the base paper and related work, Section III the methodology, Section IV the results and Section V concludes.

A. System Hardware

All development and measurement used one desktop: an Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB of RAM and integrated Intel UHD 770 graphics under Windows 11 Pro. No GPU was used; the machine was at times heavily loaded by other jobs, so timings are indicative.

B. Software Requirements

Python 3.11.9 with PyTorch 2.14.0 (CPU build) [6], NumPy 2.4.6 and NetworkX 3.4.2 [7]; FastAPI 0.115.6, SQLAlchemy 2.0.36 with SQLite, PyJWT and bcrypt; React

18.3 with Vite 6 and Node.js v24.19.0; and the Google Gemini API through the google-genai client [8].

II. LITERATURE SURVEY

Base paper. Datta, Thakkar and Dabhade [1] give a qualitative, evidence-synthesis study of GST ITC fraud. It explains why the self-assessed design invites fraud, drawing on asymmetric information, moral hazard and the Fraud Triangle, and describes how fake invoicing, circular trading and shell-company networks are carried out. Using DGGI and CBIC statistics it reports detected evasion of Rs 1.01 and Rs 2.01 lakh crore in FY 2022-23 and FY 2023-24, cases booked rising from 4,872 to 6,084, arrests from 97 to 140, and special drives against fake registrations (21,791 fake GSTINs in May-July 2023). It recommends data analytics and network analysis, dynamic AI-based risk modelling at registration, officer capacity building, faster recovery and an inter-agency intelligence grid. By its own account it relies on public sources and contains no empirical model: it names the patterns and calls for the tool, but builds and evaluates none. TaxSentinel adopts its three patterns as detection targets and implements the call for analytics, network analysis and AI-based risk modelling.

Machine learning for tax fraud. Four further papers in the batch survey apply AI and machine learning to tax compliance and income-tax fraud: an overview of AI and ML in Indian tax compliance [2], two conference papers on income-tax fraud detection [3], [4] and a boosting-based study [5]. Their full texts were not available to the authors, so, judging only by their titles and venues, they use supervised learning on labelled taxpayer records; no results are quoted from them.

Graph methods. Mehta et al. [9] study circular trading on real Telangana GST data (77.8 million invoices, 388,448 taxpayers): they learn node2vec embeddings of the sales-flow graph and cluster them to find communities of circular traders, reporting communities found rather than accuracy. Xu et al. [10] model taxation as a heterogeneous graph and propose TED, a supervised graph neural network with related-party groups, reporting F1 between 76.35 % and 94.96 % on two real tax datasets. Both target specific patterns or need labels, and neither explains its output.

Self-supervised prediction. LeCun's joint-embedding predictive architecture [11] argues for predicting in representation space; I-JEPA [12] predicts the embeddings of masked image blocks from a context block with an exponential-moving-average (EMA) target encoder, an idea from BYOL [13], and a variance term as in VICReg [14] prevents collapse. We transfer this recipe to monthly tax behaviour vectors. Anomaly detection in general is surveyed in [15].

Structure and language. Short directed cycles can be enumerated exactly with Johnson-style algorithms [16], available with a length bound in NetworkX [7]; a noisy-OR [17] combines independent causes into one probability. Language models write fluent but sometimes unsupported text [18]; retrieval-augmented generation [19] reduces this by supplying source passages. TaxSentinel supplies only computed evidence and checks the answer.

TABLE I
SURVEY OF THE MOST RELEVANT WORKS

Work	Approach	Data	Reported result	Limitation
[1]	Forensic analytics review	DGGI, CBIC statistics	Evasion Rs 1.01 to 2.01 lakh cr	No model
[2]-[5]	ML for tax fraud (per titles)	Not available	Not quoted	Supervised; full text unseen
[9]	node2vec + clustering	Telangana GST, 77.8 M invoices	Circular-trader communities	One pattern; no explanation
[10]	Heterogeneous GNN	Two real tax sets	F1 76.35-94.96 %	Supervised; no explanation
[12]	I-JEPA, masked-block prediction	Images	Strong transfer	Not applied to tax
This work	JEPA + graph + invoice + LLM	Synthetic GST	F1 0.955 (seed 42)	Synthetic; prompted LLM

Research gap. Existing GST fraud work is largely descriptive or rule-based or needs labels. Few systems learn behaviour from unlabelled tax data, combine invoice, return and network evidence in one score, recover loops and shell clusters, and explain every flag with checkable evidence. This work addresses that combination; it makes no claim to outperform supervised graph methods [10] on real data, against which it was not tested.

III. METHODOLOGY

A. Data

Real GST returns are not public, so a seeded generator produces one financial year (April 2025 to March 2026) of GST activity: 600 taxpayers in eight states and five sectors with valid-format GSTINs, 64,474 business-to-business invoices on a tiered supply network and 6,977 monthly returns (outward supplies, GSTR-2B credit available, GSTR-3B credit claimed, cash paid, filing delay). Four patterns are injected with labels: 27 shell entities in seven clusters, 19 fake-invoice buyers, 21 circular-trading firms in five rings of three to five, and 12 ITC spikes, 79 fraud taxpayers (13.2 %) and 530 fraud invoices in all. Legitimate hard negatives are included on purpose: festival seasonality, one-off capital purchases, new registrations, group companies sharing an address, late filers, credit catch-ups and reciprocal service-manufacturer trade. Labels are used only to evaluate. The generator is seeded, so seed 42 reproduces every figure reported here, and other ecosystems of 150 to 1,500 taxpayers can be generated from the application. In the bundled ecosystem 39,582 invoices carry an e-way bill and 1,071 were not reported by the seller in GSTR-1. A 100,000-row sample of PaySim [20] (134 fraud rows) serves as an out-of-domain check.

B. Features

Every active taxpayer-month becomes a 19-dimensional vector in three layers: invoice (counts, share of purchases not reported by suppliers, goods invoices without an e-way bill, round values, young suppliers), behaviour (turnover, ITC claimed, ITC-to-output-tax ratio, claim above GSTR-2B, cash share, filing delay, registration age) and network (suppliers, buyers, new partners, reciprocity, largest-buyer

share, value added). Heavy-tailed values are log-transformed and features are robust-scaled (median and IQR, clipped at ten).

C. JEPA Behaviour Encoder

Let f be an online encoder (19-64-64-32 perceptron with layer normalisation), f' an EMA copy of it without gradient, and g a predictor. For month t of taxpayer i with scaled vector z(t), previous months z(t-1..t-3) and a mask variant v, the model predicts the embedding of the whole month from the three previous embeddings (with presence flags) and from z(t) with one block hidden. Variant 0 hides everything; variants 1-3 hide the invoice, behaviour or network block, so a deviation can be attributed to a layer:

$$\hat{s} = g(f(z_{t-1}), f(z_{t-2}), f(z_{t-3}), m, f(k_v \odot z_t), \text{onehot}(v)), \quad s = f'(z_t) \tag{1}$$

$$L = \text{mean}\,(\hat{s} - s)^2 + \lambda \cdot \text{mean}_j\, \text{ReLU}(1 - \text{std}_j\, f(z)) \tag{2}$$

$$\xi \leftarrow 0.99\,\xi + 0.01\,\theta \quad \text{(after every optimiser step)} \tag{3}$$

Training uses AdamW [lr 1e-3, batch 256, 100 epochs, $\lambda = 1$] on all 6,977 taxpayer-months without labels; the saved model holds 53,420 values, trains in 11.7 s on the CPU and ends at a loss of 0.1115. At scoring time each variant's error is divided by the median error of that month, the four variants are combined by a geometric mean, the taxpayer's score is the mean of its two worst months, and a robust z-score and a logistic squash give a deviation score J in (0, 1). The behaviour-masked variant alone gives a second score B that responds to one-month ITC spikes. The training loss is not monotone: it rises from 0.012 to a peak of 0.261 at epoch 11 while the online embeddings spread out under the variance term and the EMA target follows, then falls and settles between 0.11 and 0.12 from about epoch 60.

Why JEPA. A predictive model asks whether a month is what the history and the other layers would lead one to expect. Predicting embeddings, not raw values, means the model need not reproduce the heavy-tailed noise of amounts and counts; the EMA target and the variance term prevent collapse without negative pairs; and the four masks correspond to meaningful hypotheses (history, invoice layer, behaviour layer, network layer). We did not train an isolation forest or an autoencoder on the same features, so the choice rests on design arguments, not on a measured comparison; the ablation in Section IV shows that JEPA alone is not sufficient, which is why explicit network and invoice evidence is added.

D. Network and Invoice Analysis

A directed seller-to-buyer graph is built from the invoices. Bounded cycle search (length ≤ 5) on recurring (at least two months), above-median edges finds loops, and each loop is scored as

$$\text{score}(C) = \sqrt{\text{balance}} \cdot \text{recurrence} \cdot \sqrt{\text{concentration}} \tag{4}$$

where balance is the smallest over the largest edge value, recurrence is the share of months (capped at three) in which the whole loop trades, and concentration is the loop's share of every member's purchases; overlapping loops with a score of at least 0.25 are merged into rings. Registrations sharing an address or contact number form clusters, scored from mean deviation, youth, intra-cluster trade and stopped filing. Each invoice receives a noisy-OR of eight weighted flags (unreported by the supplier, no e-way bill, round value, young supplier, supplier in a suspicious cluster, supplier later silent, loop edge, value spike) and counts as anomalous at 0.5 or more; the taxpayer's invoice score I is the peak monthly anomalous share, and the network score N combines ring, cluster and exposure evidence. A flagged taxpayer is given a named pattern from its evidence: circular trading if it belongs to a ring; shell entity if it is in a suspicious cluster and is young, has stopped filing or pays almost no cash tax; fake invoices if it buys from suspicious clusters (10 % of purchases in some month) or has at least 15 % of its purchase value flagged; ITC spike if a month's claim is at least 2.5 times its median and 1.5 times the credit available; otherwise unusual behaviour. On the bundled ecosystem 79 candidate cycles were scanned, giving 9 rings, 12 identity clusters (7 suspicious) and 16 invoice chains.

E. Risk, Evidence and Explanation

The four layer scores combine in a weighted noisy-OR, so that one strong layer suffices and agreeing layers reinforce each other; a taxpayer is flagged at 0.5:

$$R = 1 - (1 - 0.75\,J)(1 - 0.6\,B)(1 - 0.7\,N)(1 - 0.6\,I) \tag{5}$$

The weights were set on the seed-42 ecosystem. For each taxpayer the pipeline builds a numbered evidence list E1..En (layer, severity, concrete figures), names the suspected pattern and ranks taxpayers and invoice chains (each loop and each suspicious cluster with its buyers). On request a Gemini model [8] receives only the profile, scores and evidence list under a fixed system prompt: use only the evidence, cite [E#] after every claim, give no legal conclusion and write three sections (summary, why flagged, next steps) in under 320 words, at temperature 0.2. The answer is checked automatically for invalid citations, evidence and high-severity coverage and the share of its numbers found in the evidence, and is stored with the evidence it was written from. The model is therefore prompted and verified, not fine-tuned.

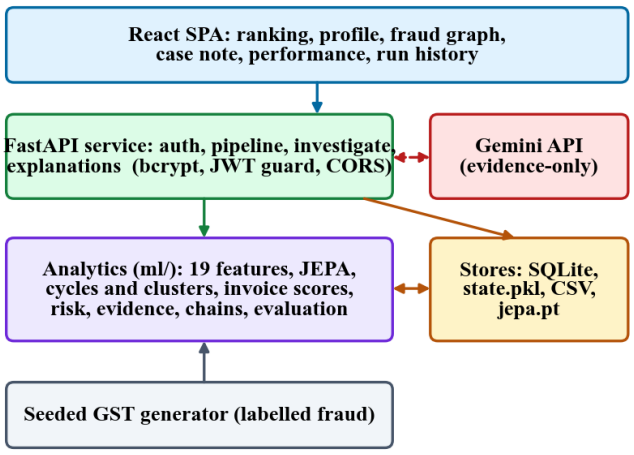


Fig. 1. Architecture of TaxSentinel: React front end, FastAPI service, analytics package, stores and the Gemini API.

F. Implementation

The backend exposes 11 endpoint families behind a JWT bearer guard (HS256, 12 h, bcrypt password hashes, pydantic validation). Generation runs on a background thread with a polled status, and the finished analytical state is held in memory and pickled, so every read endpoint is a lookup. Users, runs and stored notes are kept in SQLite. The React interface offers a ranked list, taxpayer profiles with evidence, an interactive fraud graph, invoice-chain drill-down, a case page with citation links, a performance page and a run history, in light and dark themes with keyboard shortcuts and a command palette. While a generation runs, a stage stepper and an activity log show each pipeline stage (generate, features, train, score, rings, invoices, risk, evidence) with the training epoch and loss.



Fig. 2. Main flow from GST data to a cited case note.

IV. RESULTS

A. Test Conditions

All results are on the bundled seed-42 ecosystem unless stated: 600 taxpayers (79 fraud), 64,474 invoices (530 fraud), scored without labels and then compared with the injected labels. The comparison baseline is a rule set built from standard scrutiny checks: ITC claimed over GSTR-2B by 20 % and Rs 50,000 (R1), young registration with large turnover (R2), under 1 % cash tax (R3), ITC more than tripling (R4) and stopped filing (R5). Precision@k is the share of fraud among the k highest-risk taxpayers (k = 10, 25, 50 and 79, the number of fraud taxpayers); precision, recall and F1 are taken at the flag threshold of 0.5; ROC-AUC is rank-based. An injected ring counts as recovered when one detected ring holds at least 80 % of its members. Hardware is described in Section I-A. **The data is synthetic, and the weights and threshold were chosen on this ecosystem, so these are best-case figures.**

B. Detection Performance

TaxSentinel flags 78 taxpayers, 75 correctly (3 false alarms, 4 missed): P@79 is 0.949, F1 0.955 and ROC-AUC 0.997 (TABLE II). The rule baseline flags 236, only 68 of them fraud (168 false alarms), for an F1 of 0.432. The first 50 ranked taxpayers contain no false alarm and the 10 top-ranked invoice chains are all majority-fraud. The top-ranked taxpayer (risk 0.965) belongs to ring R01, a five-party loop of 98 invoices worth Rs 44.65 Cr that passes Rs 9.78 Cr of ITC (Figs. 4 and 5).

TABLE II
DETECTION RESULTS (TAXPAYER LEVEL, THEN INVOICE LEVEL)

Method	P@79	Prec.	Recall	F1	AUC
TaxSentinel (seed 42)	0.949	0.962	0.949	0.955	0.997
Rule baseline R1-R5	0.620	0.288	0.861	0.432	0.876
JEPA + behaviour only	0.481	0.842	0.203	0.327	0.843
Graph + invoice only	0.848	1.000	0.798	0.887	0.940
TaxSentinel, 5 other seeds (mean)	0.888	0.924	0.857	0.888	0.988
Rule baseline, 5 other seeds (mean)	0.625	-	-	0.457	0.877
Invoice score >= 0.5	-	0.883	0.968	0.924	0.9997
Invoice not in supplier GSTR-1	-	0.120	0.242	0.160	-

C. Ablations and Patterns

The ablations are the most informative result (Fig. 3). The JEPA with the behaviour variant alone reaches an F1 of only 0.327: precision 0.842 but recall 0.203, because the 27 look-alike shell firms form their own "normal". Graph plus invoice evidence reaches 0.887 with precision 1.000, and the full model 0.955; removing the invoice layer leaves P@79 unchanged at 0.949. The JEPA is thus a useful but partial signal that mainly adds recall, and most detection power comes from the network and invoice layers. Recall at the threshold is 1.00 for circular trading (rules 0.86), 1.00 for shell entities (rules 1.00), 0.89 for fake-invoice buyers (rules 0.58) and 0.83 for ITC spikes, where **the rule baseline is better** (1.00): a claim that triples in one month is exactly what rule R4 tests, while the model's score dilutes a single month. All five injected rings were recovered.

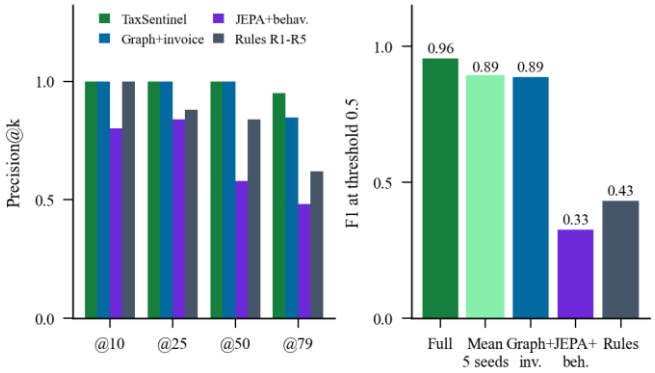


Fig. 3. Left: precision@k of the full model, two ablations and the rule baseline. Right: F1 at the 0.5 threshold (seed 42, and the mean over five other seeds).

TABLE III
RECALL BY INJECTED PATTERN AT THE FLAG THRESHOLD (SEED 42)

Pattern	Count	TaxSentinel	Rules	Named
Circular trading	21	1.00	0.86	1.00
Shell entity	27	1.00	1.00	1.00
Fake-invoice buyer	19	0.89	0.58	1.00
ITC spike	12	0.83	1.00	1.00

D. Generalisation, Invoices and Transfer

Re-running the whole pipeline on five other generated ecosystems of 600 taxpayers (seeds 1, 2, 3, 7, 11) with the same weights and threshold gives a mean F1 of 0.888 (std 0.052, range 0.790-0.933), a mean ROC-AUC of 0.988 (range 0.970-0.995) and a mean precision@k of 0.888,

against a rule-baseline mean F1 of 0.457 (range 0.412-0.498). Ranking quality is therefore stable and the advantage over rules holds on every seed, but the F1 at the fixed 0.5 threshold is lower and less stable than the seed-42 figure, so the headline numbers are a best case. The loss is concentrated in two patterns: recall ranges from 0.33 to 0.81 for fake-invoice buyers and from 0.42 to 0.92 for ITC spikes, while circular trading stays at 1.00-1.00, shell entities at 0.96-1.00 and every injected ring is recovered. At invoice level, the invoice score reaches F1 0.924 (precision 0.883, recall 0.968, 581 flagged) against 0.160 for the classic check that a purchase is missing from the supplier's GSTR-1 (1,071 flagged). Sweeping the threshold on the seed-42 scores gives F1 of 0.939, 0.955 and 0.933 at 0.4, 0.5 and 0.6, but only 0.813 at 0.3 (113 flagged) and 0.838 at 0.7 (57 flagged): the choice of threshold matters. As a transfer check the same JEPA module, trained without labels on 100,000 PaySim transactions for 6.2 s, reaches a ROC-AUC of 0.935 against 0.790 for ranking by amount (precision@100 0.31 against 0.02).

E. Explanations, Testing and Speed

Of 10 planned case notes, 6 completed before the free Gemini quota and availability limits (HTTP 429, 503 and 504) stopped the rest. Over these 6 notes there were 0 % invalid citations, 10.8 citations per note, 100 % evidence coverage, 100 % high-severity coverage and 84.3 % numeric grounding; the ungrounded numbers are mostly derived figures (a total of two evidence amounts, a rounded share) that the simple check cannot match. The same case written 4 times during development gave numeric grounding between 0.87 and 0.90, because the model is not deterministic even at temperature 0.2. These checks measure grounding, not the correctness of the advice, and no human judged the notes.

An end-to-end smoke test (registration, sign-in, two regenerations, every endpoint family and one real Gemini call) passed 20 of 20 checks in 39.4 s. Black-box checks sent invalid, forged and hostile requests (wrong password, unknown user, duplicate registration, short password, malformed e-mail or JSON, a token with an altered signature, every protected endpoint without a token, SQL-injection-style and script-tag search text, out-of-range paging and generation parameters, unknown identifiers and a second generation while one was running); the expected 401, 404, 409 or 422 status was returned in 41 of 41 cases. Median read-endpoint latency is 1.7-12.1 ms (20 sequential calls each, single client) and a full generate-train-analyse run takes 18.6 to 29.0 s for 600 taxpayers (longer when other jobs load the machine); the API process held about 0.5 GB with the 600-taxpayer state loaded. There is no unit-test suite, load test or user study.

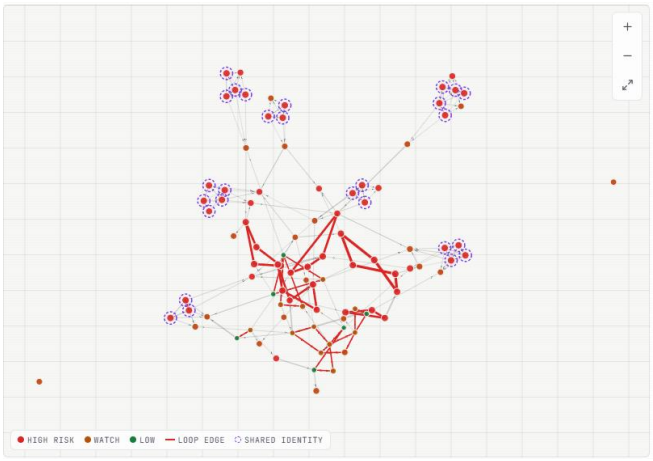


Fig. 4. Fraud network of the seed-42 ecosystem: red edges are detected circular-trading loops, ringed nodes are shared-identity members.

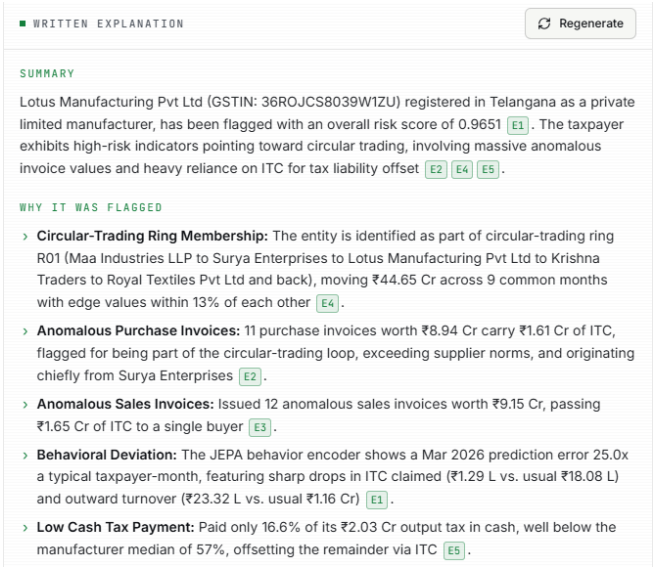


Fig. 5. Case note for the top-ranked taxpayer; each claim ends in an [E#] chip linked to its evidence.

F. Comparison with the Base Paper

The base paper [1] reports no model, accuracy or other measured performance, so no like-for-like numerical comparison is possible. Qualitatively, TaxSentinel implements what it recommends: the three fraud patterns as detection targets, analytics and network analysis, and AI-based risk modelling (applied to existing taxpayers, not at registration). Against the rule checks that the paper's setting implies, the gain on the same data is F1 0.955 against 0.432 on seed 42 and 0.888 against 0.457 on average over five other seeds. For context only, TED [10] reports F1 of 76.35-94.96 % on two real datasets with supervised training; the data, task and supervision differ, so the figures are not comparable.

G. Failure Cases and Observed Limits

The 3 false alarms are not loops: they are legitimate wholesalers and manufacturers flagged on behavioural deviation alone (JEPA scores 0.37-0.80, network score 0, no ring or cluster), so the JEPA's own false positives are where a

single signal misleads. The 4 misses all lie below the threshold but within the top 21 % of the ranking (ranks 80 to 122 of 600): two ITC-spike cases whose single-month spikes are diluted in the deviation score (risk 0.41 and 0.47) and two fake-invoice buyers with weak network (0.31) and invoice (0.11-0.12) evidence (risk 0.29 and 0.32). ITC-spike recall (0.833) is below the rule baseline's (1.00). Separately, two of the nine detected rings, the two with the lowest scores (0.30 and 0.27), contain no injected-fraud taxpayer: they are legitimate reciprocal trade loops that the generator includes on purpose, so a loop is a lead, not proof. Other limits are the synthetic data, weights and threshold tuned on one seed, the absence of a held-out split for the encoder (the cross-seed study is the nearest substitute), and the dependence on an external language-model quota.

V. CONCLUSION

TaxSentinel shows that GST ITC-fraud detection can be built without fraud labels and still explain itself. A 53,420-value JEPA learns normal taxpayer behaviour in 11.7 s on a CPU; combined with loop and cluster detection and invoice scoring, it ranks the synthetic ecosystem with precision@79 of 0.949 and ROC-AUC 0.997, recovers all five injected rings, scores invoices with F1 0.924, and supports case notes with no invalid citations in the six evaluated. The key finding is that several layers beat one: the JEPA alone reaches only F1 0.327, and its value lies in adding recall to network and invoice evidence. The honest limits are the synthetic data, a fixed threshold whose F1 is lower on other generated ecosystems, a prompted rather than fine-tuned language model evaluated on six notes, and no unit, load or user testing. The output is a risk indication for review, never a finding of fraud.

Future work is, in order of importance: calibrating weights on a small set of confirmed real cases, validation on anonymised real returns with a tax administration, head-to-head baselines against an isolation forest and an autoencoder, role-based access with case management, scaling the graph analysis to state-sized data, and a human evaluation of the explanations.

REFERENCES

[1] S. Datta, H. Thakkar and S. Dabhade, "Combating GST input tax credit fraud in India: Forensic accounting, data analytics, and legal enforcement strategies," *Int. J. Manage., Public Policy Res.*, vol. 5, no. 1, pp. 37-46, Jan.-Mar. 2026, doi: 10.55829/ykz3ea41.

[2] "The role of artificial intelligence and machine learning in enhancing tax compliance and fraud detection in India," *J. Informatics Educ. Res.*, 2025.

[3] "Income tax fraud detection using machine learning," in *Proc. 8th Int. Conf. Trends Electron. Informatics (ICOEI)*, 2025.

[4] "Income tax fraud detection system using AI and machine learning," in *Proc. Int. Conf. Comput. Commun. (COMPUTINGCON)*, 2025.

[5] "Boosting-based machine learning for efficient income tax fraud detection," *Int. J. Res. Trends Innov.*, 2026.

[6] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 8024-8035.

[7] A. A. Hagberg, D. A. Schult and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python Sci. Conf. (SciPy)*, 2008, pp. 11-15.

[8] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," arXiv:2312.11805, 2023.

[9] P. Mehta, S. Bhargava, M. R. Kumar, K. S. Kumar and C. S. Babu, "Representation learning on graphs to identifying circular trading in Goods and Services Tax," arXiv:2208.07660, Aug. 2022.

[10] Y. Xu, B. Shi, B. Dong, J. Wang, H. Wei and Q. Zheng, "TED: Related party transaction guided tax evasion detection on heterogeneous graph," arXiv:2605.26984, May 2026.

[11] Y. LeCun, "A path towards autonomous machine intelligence," version 0.9.2, OpenReview, Jun. 2022.

[12] M. Assran *et al.*, "Self-supervised learning from images with a joint-embedding predictive architecture," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 15619-15629.

[13] J.-B. Grill *et al.*, "Bootstrap your own latent: A new approach to self-supervised learning," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 21271-21284.

[14] A. Bardes, J. Ponce and Y. LeCun, "VICReg: Variance-invariance-covariance regularization for self-supervised learning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[15] V. Chandola, A. Banerjee and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1-58, 2009.

[16] D. B. Johnson, "Finding all the elementary circuits of a directed graph," *SIAM J. Comput.*, vol. 4, no. 1, pp. 77-84, 1975.

[17] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Mateo, CA, USA: Morgan Kaufmann, 1988.

[18] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1-38, 2023.

[19] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459-9474.

[20] E. A. Lopez-Rojas, A. Elmir and S. Axelsson, "PaySim: A financial mobile money simulator for fraud detection," in *Proc. 28th Eur. Model. Simul. Symp. (EMSS)*, 2016.